Tópicos en Implementación: Aprendizaje Profundo

Alvaro J. Riascos Villegas

Uniandes y Quantil

Noviembre de 2026

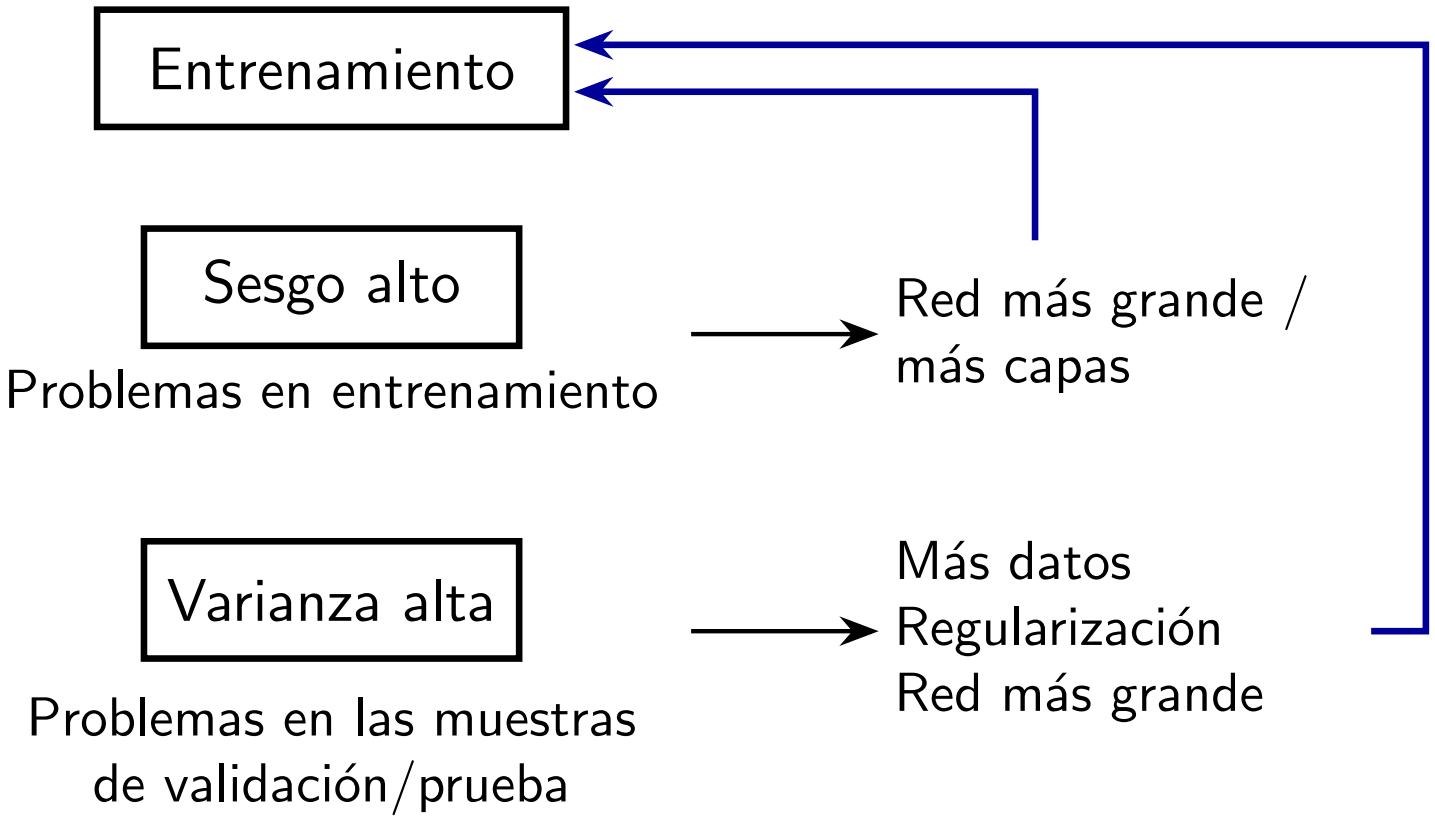
Contenido

1. Entrenamiento y regularización
   - Entrenamiento
   - Regularización
   - Doble descenso y sesgo inductivo

Entrenamiento: datos

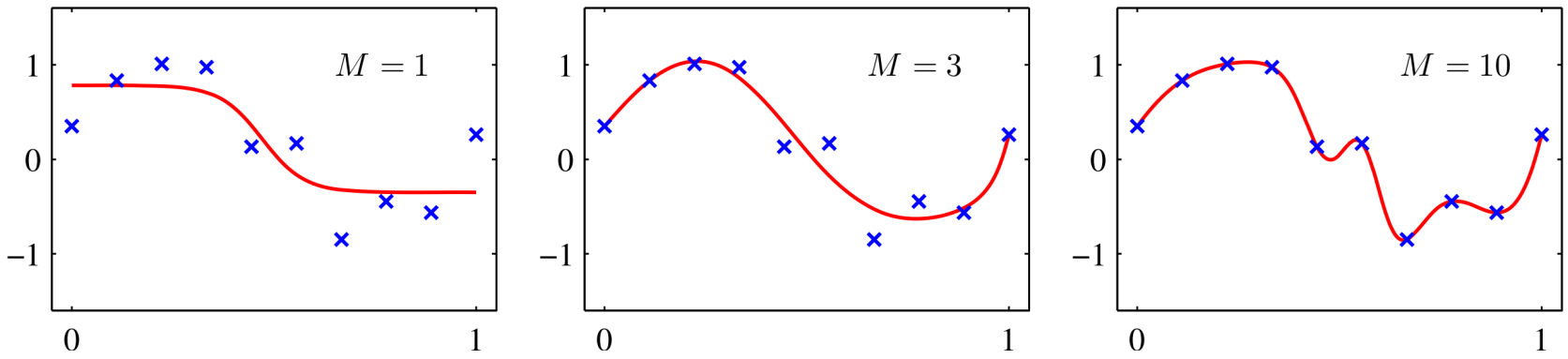
- En proporción, las muestras de validación y de prueba son cada vez más pequeñas.
- Tradicionalmente, una muestra se separaba en 70 % entrenamiento, 15 % validación (desarrollo, *dev*) y 15 % prueba.
- Con la gran cantidad de información disponible actualmente, es más común una partición de 96 % entrenamiento, 2 % validación y 2 % prueba.

Entrenamiento: análisis



Que una red más grande pueda reducir la varianza se relaciona con el doble descenso; véase Fernández-Villaverde (2025).

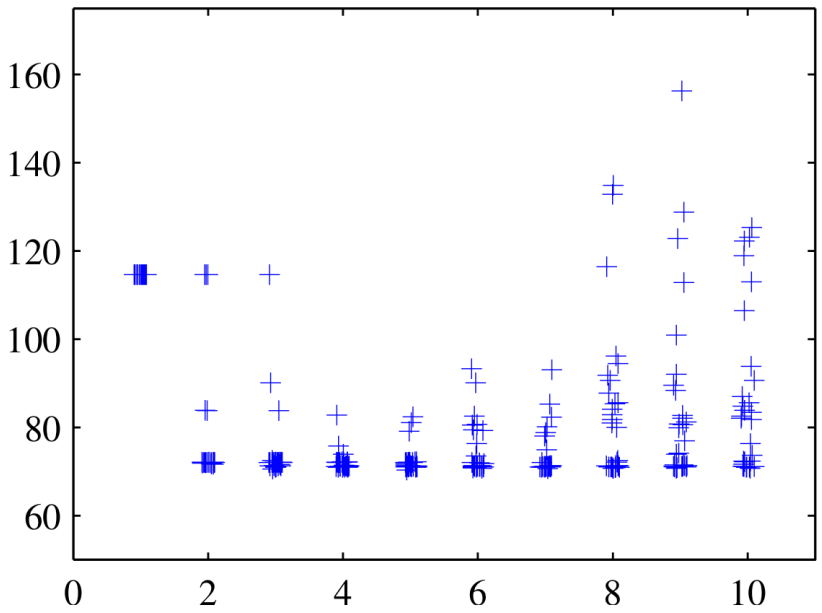
Entrenamiento: sesgo vs. varianza



- Redes de dos capas entrenadas con 10 puntos del conjunto de datos sinusoidal.
- Se ajustan redes con $M = 1$, 3 y 10 neuronas ocultas, minimizando la suma de errores al cuadrado con un algoritmo de gradiente conjugado escalado.
- $M = 1$: subajuste (sesgo alto); $M = 10$: sobreajuste (varianza alta).

Fuente: Bishop (2006), fig. 5.9.

Entrenamiento: óptimos locales



- Error de prueba (suma de errores al cuadrado) en el conjunto de datos polinomial, según el número de neuronas ocultas.
- Para cada tamaño de red se hacen 30 inicializaciones aleatorias: los pesos iniciales se muestrean de una normal isotrópica con media cero y varianza 10.
- La dispersión entre inicializaciones muestra el efecto de los mínimos locales.

Fuente: Bishop (2006), fig. 5.10.

Entrenamiento: aumento de datos (*data augmentation*)

- Para aumentar la cantidad y la variedad de los datos se transforman los ejemplos existentes:
  - rotaciones, traslaciones y reflexiones;
  - acercamientos (*zoom*);
  - ruido.

Regularización: decaimiento de pesos (*weight decay*)

- La función de costo regularizada es

$$J_R(W, b) = J(W, b) + \frac{\lambda}{2m} \sum_{l=1}^{L} \|W^{[l]}\|_F^2,$$

donde (W, b) abrevia $(W^{[1]}, b^{[1]}, \ldots, W^{[L]}, b^{[L]})$, J es la función de costo sin regularización y $\|W\|_F^2 = \sum_{i,j} w_{ij}^2$ (norma de Frobenius).

- En la retropropagación basta con reemplazar el gradiente por

$$dW_R^{[l]} = dW^{[l]} + \frac{\lambda}{m} W^{[l]},$$

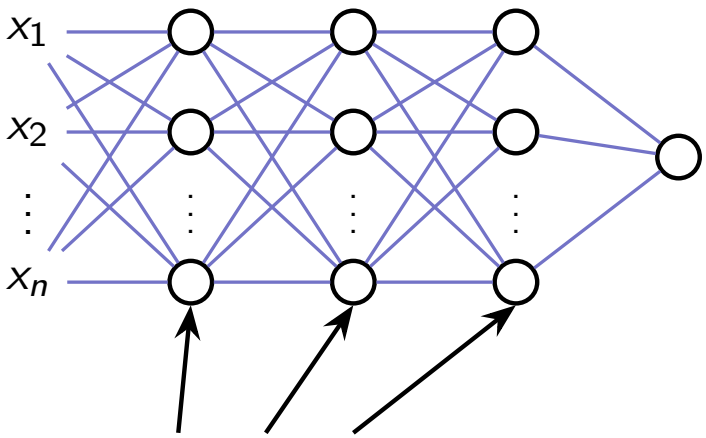
de modo que la actualización es

$$W^{[l]} \leftarrow W^{[l]} - \alpha\, dW_R^{[l]} = \left(1 - \frac{\alpha\lambda}{m}\right) W^{[l]} - \alpha\, dW^{[l]}.$$

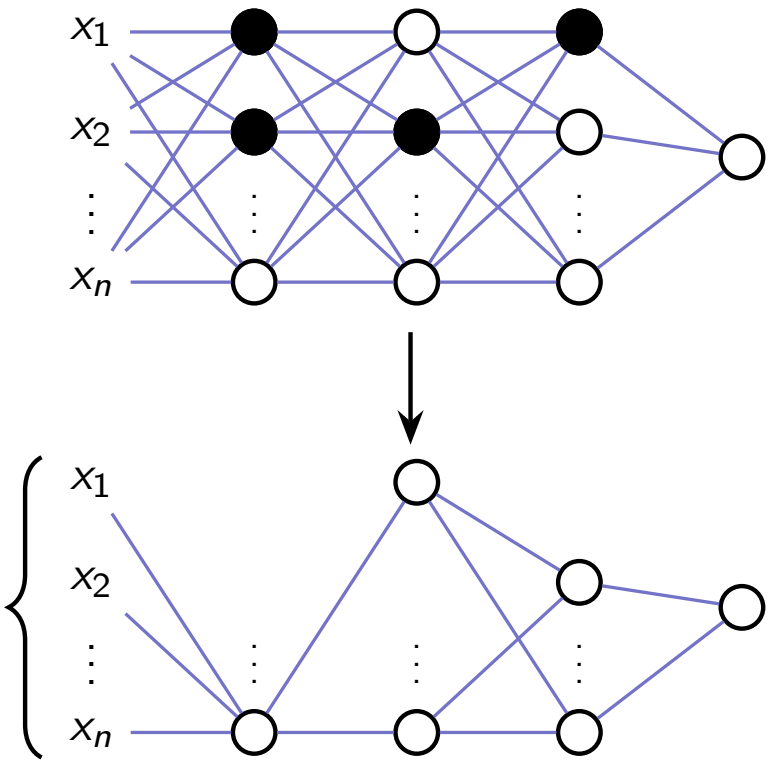
Regularización: decaimiento de pesos (*weight decay*)

- El factor $(1 - \alpha\lambda/m) < 1$ encoge los pesos en cada iteración: de ahí el nombre *weight decay*.
- La regularización controla la complejidad del modelo.
- Intuitivamente: si λ aumenta, los pesos se hacen pequeños, $\|z^{[l]}\|$ es pequeño y, con activaciones como tanh o la sigmoide, cada capa opera en la zona aproximadamente lineal: la red se comporta como un modelo más simple.

Regularización: *dropout*



Se conserva cada neurona con probabilidad p



Menos pesos para entrenar

Regularización: *dropout*

- Sea p la probabilidad de conservar cada neurona. En cada iteración se generan $d_j^{[l]} \sim \text{Bernoulli}(p)$ independientes y se reescala:

$$\tilde{a}^{[l]} = \frac{d^{[l]} \odot a^{[l]}}{p}.$$

- Así $\mathbb{E}\left[\tilde{a}^{[l]}\right] = a^{[l]}$: la siguiente capa,

$$z^{[l+1]} = W^{[l+1]}\tilde{a}^{[l]} + b^{[l+1]},$$

recibe en promedio la misma entrada que sin *dropout* (*dropout* invertido).

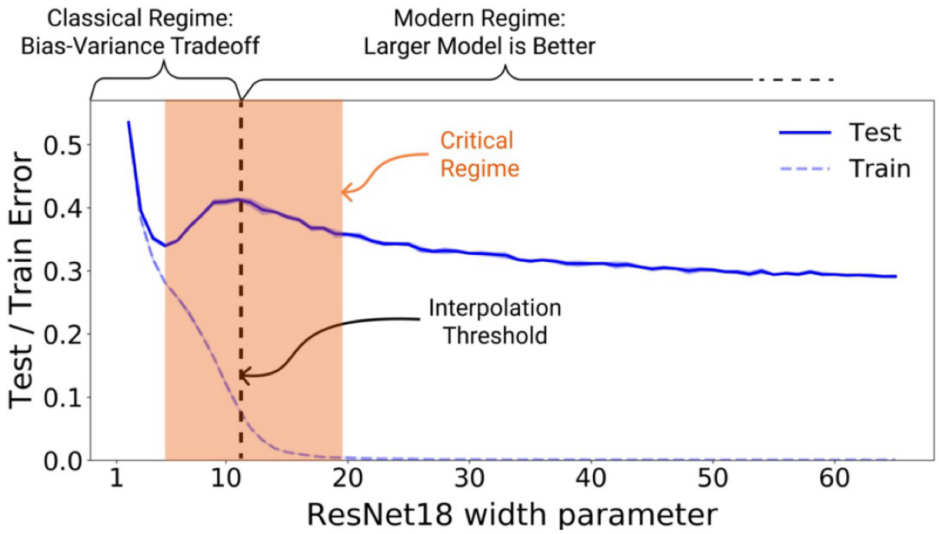
Regularización: *dropout*

- *Dropout* distribuye la importancia entre las neuronas e impide que una tenga demasiado peso (Srivastava et al., 2014).
- En principio, cada capa puede tener una probabilidad distinta de *dropout*.
- Al hacer predicciones se usa la red completa, sin *dropout*; gracias al reescalamiento no hay que ajustar los pesos.
- El entrenamiento puede tardar un poco más, pues las actualizaciones son ruidosas.
- Como la función de costo no decrece monótonamente, es buena práctica verificar primero el entrenamiento sin *dropout*.

Regularización: parada temprana (*early stopping*)

- Se detiene el entrenamiento cuando el error en la muestra de validación deja de disminuir.
- El problema es que es una estrategia que mezcla dos problemas: la minimización de la función de costo y evitar el sobreajuste del modelo.
- La alternativa, regularizar explícitamente y escoger λ con una muestra de validación, es computacionalmente más costosa.
- Con modelos grandes, el error de prueba puede volver a bajar con más épocas (doble descenso en función de las épocas, más adelante).

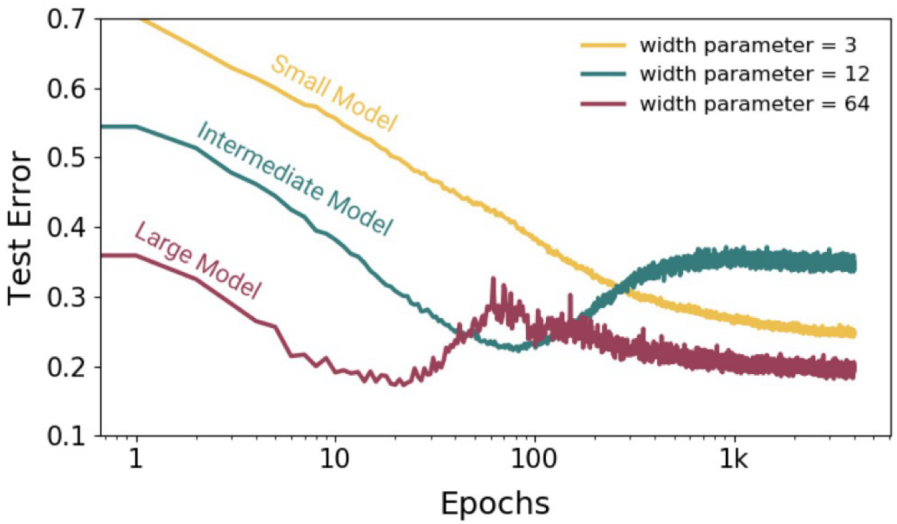
Doble descenso en función de la complejidad



- Errores de entrenamiento y de prueba de una ResNet18 en clasificación de imágenes. Eje horizontal: hiperparámetro de ancho, que controla el número de neuronas ocultas y, con ello, el de pesos y sesgos.
- Línea punteada vertical: umbral de interpolación, la complejidad a partir de la cual el modelo puede, en principio, alcanzar error cero en entrenamiento.
- Régimen clásico: curva en U del dilema sesgo-varianza, con un máximo cerca del umbral. Régimen moderno: más allá del umbral, aumentar la complejidad vuelve a reducir el error de prueba (Belkin et al., 2019; Nakkiran et al., 2021).

Fuente: Bishop and Bishop (2024), fig. 9.9, a partir de Nakkiran et al. (2021).

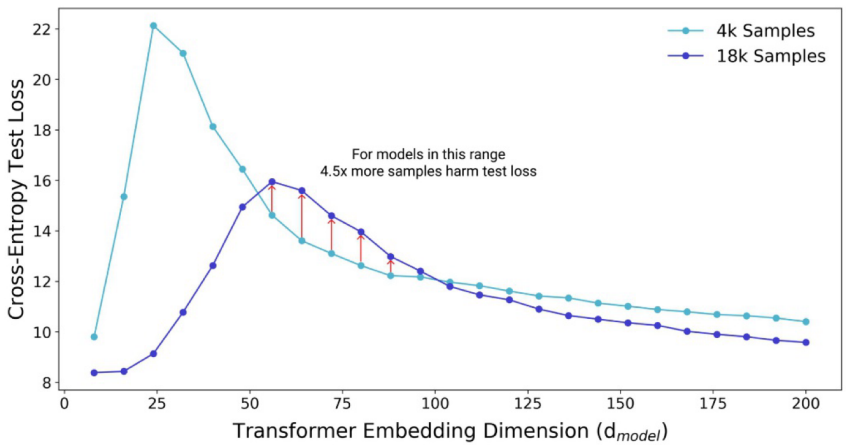
Doble descenso en función de las épocas



- Error de prueba según el número de épocas de descenso de gradiente, para modelos ResNet18 de tres tamaños (parámetro de ancho 3, 12 y 64).
- La complejidad efectiva del modelo aumenta con el número de épocas.
- El doble descenso se observa solo en el modelo suficientemente grande.

Fuente: Bishop and Bishop (2024), fig. 9.10, a partir de Nakkiran et al. (2021).

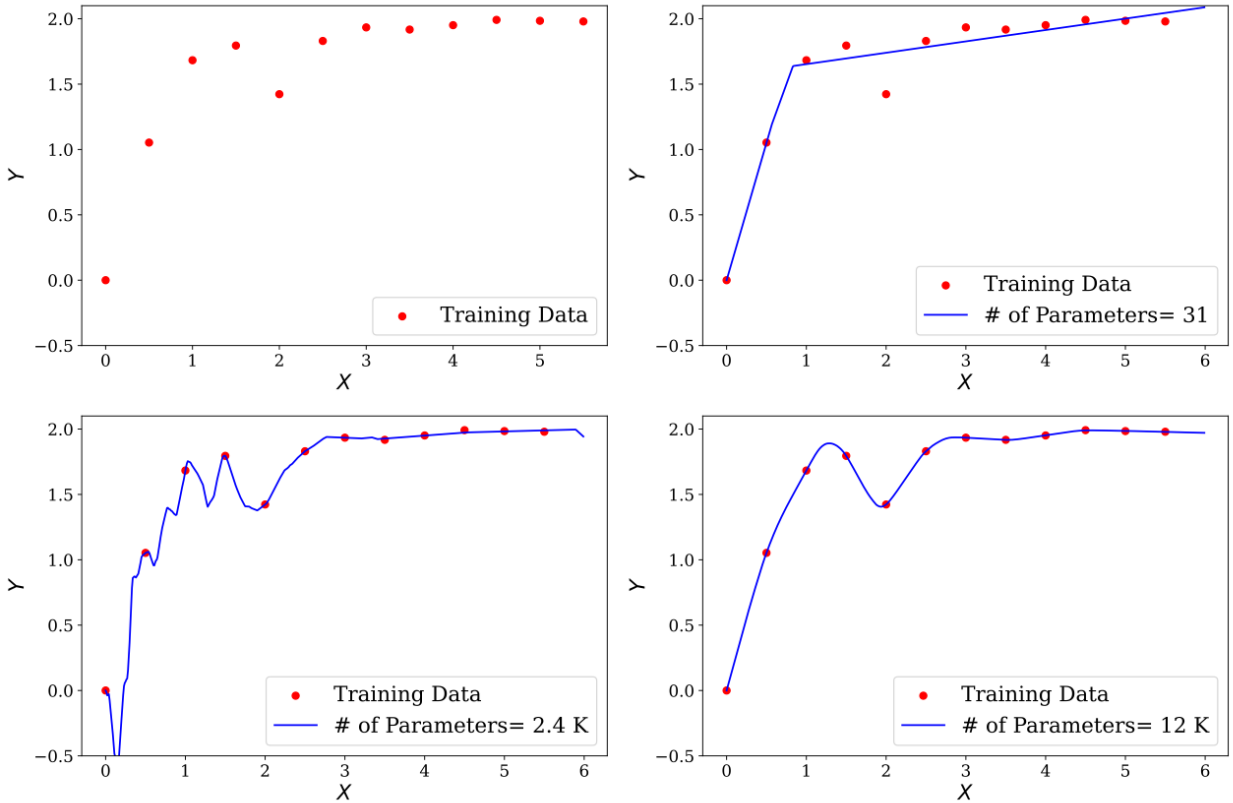
Doble descenso en función del número de ejemplos



- Error de prueba (entropía cruzada) de un *transformer* grande según la dimensión del *embedding*, que controla el número de parámetros.
- Aumentar la muestra de entrenamiento de 4.000 a 18.000 ejemplos en general reduce el error de prueba. . .
- . . . pero, para algunos valores intermedios de complejidad, lo aumenta (flechas rojas).

Fuente: Bishop and Bishop (2024), fig. 9.11, a partir de Nakkiran et al. (2021).

Sesgo inductivo



Interpolación con una red neuronal sobreparametrizada (12 puntos de entrenamiento; redes con 31, 2.401 y 12.001 parámetros).

Fuente: Fernández-Villaverde (2025), fig. 13.

Sesgo inductivo: interpretación

- Arriba a la izquierda: 12 puntos de entrenamiento generados por $Y = 2\left(1 - e^{-|X+\sin(X^2)|}\right)$.
- 31 parámetros: la red captura la forma general, pero no tiene capacidad para interpolar todos los puntos.
- 2.401 parámetros (umbral de interpolación): ajusta perfectamente los datos, pero es "ondulada" entre los puntos.
- 12.001 parámetros: también interpola, pero de forma suave, más "plana".
- Sesgo inductivo: entre todas las soluciones que interpolan, los optimizadores basados en gradientes eligen las más planas (regularización implícita).
- Todas las redes tienen una capa oculta con activación ReLU y se entrenan con el optimizador LBFGS.

Fuente: Fernández-Villaverde (2025).

Referencias

Belkin, M., Hsu, D., Ma, S., and Mandal, S. (2019). Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer, New York.

Bishop, C. M. and Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer, Cham.

Fernández-Villaverde, J. (2025). Deep learning for solving economic models. PIER Working Paper 25-017, Penn Institute for Economic Research, University of Pennsylvania.

Nakkiran, P., Kaplun, G., Bansal, Y., Yang, T., Barak, B., and Sutskever, I. (2021). Deep double descent: Where bigger models and more data hurt. *Journal of Statistical Mechanics: Theory and Experiment*, 2021(12):124003.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958.